



Universidad Nacional de Asunción
Facultad Politécnica
Ingeniería en Informática
Propuesta de Proyecto Final de Carrera

**Diseño de un proceso integrado de aseguramiento de
calidad (QA) en entornos de despliegue automatizado
(DevOps)**
Enrique G. Sánchez Caballero

TUTOR: Prof. Ing. Ellen Lujan Mendez Xavier, Mag.

2026

1. SÍNTESIS DEL PROYECTO

1.1 TÍTULO INICIAL DEL PROYECTO

Diseño de un proceso integrado de aseguramiento de calidad (QA) en entornos de despliegue automatizado (DevOps)

SINTETIZAR EL PROYECTO EN TRES PALABRAS CLAVE

Automatización

Métricas

Pipeline CI/CD

1.2 DURACIÓN DEL PROYECTO

12

MESES

1.3 DATOS DE LOS ESTUDIANTES Y DEL ASESOR DEL PROYECTO

NOMBRE DEL ASESOR: ELLEN LUJAN MENDEZ XAVIER	
Email: emendez@pol.una.py	Teléfono: 595991805937
CARGO: Profesor Orientador	FIRMA:

NOMBRE DEL ALUMNO: Enrique Gabriel Sánchez Caballero	
CI: 5788077	PLAN DE ESTUDIOS: 2008
Email: enriquecaballero01@fpuna.edu.py	Teléfono: +595 986 129386
AÑO DE INGRESO: 2017	FIRMA:

RESUMEN DEL PROYECTO

El desarrollo moderno de software requiere de procesos rigurosos de Aseguramiento de Calidad (QA) para garantizar que los productos desarrollados cumplan con los requerimientos establecidos, se minimicen los defectos y se mantenga una calidad alta durante todo el ciclo de desarrollo del software. Según el ISTQB (International Software Testing Qualifications Board), el QA es esencial no solo para encontrar bugs sino entender como el testing reduce riesgos y valida que el software cumpla con las necesidades de los usuarios y del negocio. Se destaca que el testing debe iniciar en épocas tempranas y debe ser establecido continuamente en todas las fases del desarrollo, detectando errores lo antes posible, lo cual ayuda a ahorrar recursos y aumentar la confianza en el producto. [1]

DevOps es un enfoque de Desarrollo de software que integra a los equipos de Desarrollo y Operaciones en una entidad unificada para optimizar la entrega de software y mejorar la colaboración. Fomenta la integración y entrega continua y especialmente la automatización a lo largo de todo el ciclo de vida del producto, lo cual alivia las limitaciones de los métodos tradicionales y hasta de los ágiles. Uno de los aspectos más importantes de esta práctica es la implementación del CI/CD (Integración Continua/Entrega Continua), que automatiza la compilación, las pruebas y el desarrollo del software. Esto permite que los cambios hechos en el código se integren continuamente y se prueben automáticamente, permitiendo lanzamientos rápidos y confiables con una mínima intervención manual. [2]

La integración de procesos de QA dentro de entornos DevOps y *pipelines CI/CD* representa el siguiente paso hacia un proceso de entrega de software más confiable, ágil y eficiente. Al incorporar prácticas de aseguramiento de calidad desde las primeras etapas y automatizar pruebas dentro del *pipeline*, se garantiza que cada cambio pase por validaciones continuamente antes de ser lanzado. Esta combinación no solo acelera la detección de defectos, sino que asegura también que la calidad no se vea afectada por la velocidad de entrega. En el marco de este trabajo, se busca demostrar cómo la integración estratégica de QA en flujos DevOps, apoyado en CI/CD, puede reducir riesgos, optimizar recursos y entregar productos de alta calidad de forma consistente, alineando así los objetivos técnicos con las necesidades del negocio y la satisfacción del usuario final.

1.4 OBJETIVOS DEL PROYECTO.

1.4.1 OBJETIVO GENERAL

Proponer un proceso integrado de aseguramiento de calidad (QA) en un entorno DevOps, enfocado en la automatización de pruebas dentro del pipeline CI/CD, para optimizar la calidad del software y garantizar la detección temprana de defectos.

1.4.2 OBJETIVOS ESPECÍFICOS

1. OBJETIVO 1

Identificar las mejores prácticas y herramientas para la automatización de pruebas y la medición de calidad en DevOps, basándose en una revisión bibliográfica exhaustiva.

2. OBJETIVO 2

Identificar riesgos de calidad y ambigüedades funcionales mediante el análisis estático de los requerimientos del sistema.

3. OBJETIVO 3

Implementar herramientas de análisis estático de código (SAST) para la detección automatizada y temprana de vulnerabilidades.

4. OBJETIVO 4

Diseñar y desarrollar un conjunto de pruebas automatizadas que cubran la funcionalidad crítica del sistema.

5. OBJETIVO 5

Integrar pruebas automáticas en el pipeline CI/CD, asegurando su ejecución continua con cada cambio introducido en el código.

6. OBJETIVO 6

Definir e implementar métricas que permitan medir la eficiencia del proceso de QA y la estabilidad del producto.

7. OBJETIVO 7

Evaluar los resultados obtenidos para identificar cuellos de botella, áreas de riesgo y oportunidades de mejora continua en el flujo de trabajo.

2. FORMULACIÓN GENERAL DEL PROYECTO

2.1 PROBLEMA E HIPÓTESIS

2.1.1 ¿CUÁL ES EL PROBLEMA U OPORTUNIDAD?

En el desarrollo moderno de software actual, existe la necesidad de lanzar software rápidamente sin comprometer la calidad de este. Vivimos en una sociedad donde el tiempo es dinero, y el éxito de un producto puede verse afectada por la reputación que las personas tienen sobre este. Es por eso por lo que es absolutamente necesario evitar que bugs aparezcan en el entorno de producción, donde los usuarios finales tienen acceso a este. Los bugs pueden tener un impacto leve, como por ejemplo una experiencia de usuario afectada, a un impacto gravísimo, donde se pierden no solo cantidades enormes de dinero tanto para la empresa como para los usuarios, sino también algo hasta más importante: reputación. [3]

2.1.2 ¿CUÁL ES LA SOLUCIÓN PROPUESTA POR EL PROYECTO DE INVESTIGACIÓN?

La solución consiste en implementar un marco de "Calidad Continua" (Continuous Quality) dentro del ciclo de vida DevOps. Este marco se materializa en un pipeline CI/CD que integra y automatiza las validaciones de calidad. A diferencia de un enfoque tradicional, esta propuesta abarca:

- La ejecución automática de una suite de pruebas (unitarias, integración y funcionales). Orientada a validar la funcionalidad crítica del sistema y alineada con el diseño de pruebas automatizadas.
- La integración de herramientas de análisis estático de requerimientos y de código (SAST) para la identificación temprana de riesgos de calidad, ambigüedades funcionales y vulnerabilidades, contribuyendo a la detección anticipada de defectos.
- La recolección de métricas para la toma de decisiones. De esta forma, se busca validar sistémicamente cada cambio, permitiendo prevenir defectos antes de que lleguen a producción.
- La evaluación sistemática de los resultados obtenidos a partir de las ejecuciones del pipeline, permitiendo identificar cuellos de botella, áreas de riesgo y oportunidades de mejora continua dentro del flujo DevOps.

2.1.3 ¿CUÁL ES LA HIPÓTESIS DEL PROYECTO?

La hipótesis es que la implementación del marco de "Calidad Continua" permitirá no solo la detección temprana de defectos, sino también la cuantificación objetiva de la calidad tanto del proceso como del producto. Este enfoque sistemático y basado en métricas generará un aumento en la confianza para introducir cambios y demostrará que la inversión inicial en la automatización de QA se traduce directamente en una mayor eficiencia del proceso, estabilidad del sistema y calidad del producto final.

2.2 ANTECEDENTES

DevOps representa un cambio fundamental en la cultura de desarrollo de software, reduciendo la distancia entre los equipos de desarrollo y operación, promoviendo la automatización, la colaboración y el mejoramiento continuo a través del ciclo de vida del desarrollo de software [7]. La integración de procesos de QA dentro de DevOps, particularmente en los *pipelines* de CI/CD, está emergiendo como un área de investigación crítica para organizaciones que desean mantener una alta calidad de software y al mismo tiempo acelerando los ciclos de lanzamiento [8].

El objetivo principal de DevOps es incrementar la velocidad de los despliegues, las frecuencias y la calidad a través de la automatización de los procesos de lanzamiento [7]. Este cambio de paradigma requiere una reevaluación de los procesos de QA tradicionales, cambiando las pruebas manuales al final del ciclo de desarrollo hacia pruebas automáticas, continuas, integradas a través de todo el *pipeline* [9].

Estudios recientes muestran que la investigación de DevOps y la calidad de software se ha centrado principalmente en la automatización, la cultura, la entrega continua y la retroalimentación rápida [7]. Sin embargo, existen todavía huecos importantes de investigación en áreas como medición, desarrollo de métricas en diferentes etapas para juzgar el rendimiento, cultura, prácticas para asegurar la calidad y factores de calidad como usabilidad, eficiencia, mantenimiento del software y portabilidad [7].

Pipelines modernos de CI/CD implementan *pipelines* de despliegue que modela el proceso de entrega, dividiendo la entrega de software en varias etapas, cada una de estas están diseñadas para verificar la calidad desde distintas perspectivas [9]. Estos *pipelines* deben proveer ciclos de retroalimentación rápida mientras que al mismo tiempo proveer visibilidad al flujo constante de cambios [9].

La automatización rigurosa de pruebas fue identificada como un factor principal para la entrega continua, particularmente para negocios online que despliegan cambios hasta varias veces por día [10]. El caso de estudio de una compañía Austriaca demuestra que para lograr *pipelines* de entrega continua efectivos se requiere sofisticadas estrategias de pruebas automáticas para abordar desafíos técnicos y organizacionales [10]. Normalmente, implementaciones efectivas abarcan los siguientes puntos:

- Pirámide de Pruebas: se hace énfasis en pruebas unitarias en la base, seguido de pruebas de integración y pruebas *end-to-end* mínimas.
- Ejecución de pruebas en paralelo: correr pruebas en paralelo para reducir el tiempo de retroalimentación.
- Gestión de entornos de prueba: configuración automática de ambientes de prueba.
- Gestión de datos de prueba: estrategias para generar y gestionar datos de prueba.

Un estudio sobre proyectos *open source* usando *Travis CI* revela que herramientas de análisis estático son integrados comúnmente en *pipelines CI*, enfocándose principalmente en asegurar que el código se adhiera a estándares comunes y al cumplimiento de licencias [11]. Curiosamente, los fallos de compilación relacionados a herramientas que identifican posibles errores o vulnerabilidades ocurren con poca frecuencia, y en algunos casos, estas herramientas son usadas en un modo más “suave” sin causar fallos de compilación [11]. El estudio encontró que las fallas de compilación debido a herramientas de análisis estático son rápidamente arregladas cuando se resuelven los problemas de fondo y no simplemente desactivando las advertencias, y usualmente estos arreglos son documentados apropiadamente [11]. Esto sugiere que la integración de análisis estático puede mejorar significativamente la calidad del código cuando es implementada apropiadamente.

Uno de los mayores desafíos en la implementación de DevOps es la evaluación del rendimiento [12]. Investigaciones recientes introdujeron atributos de calidad basados en mediciones y análisis comparativos con métodos tradicionales para evaluar el rendimiento de DevOps [12]. Las métricas clave para evaluar son:

- Frecuencia de despliegue
- Tiempo de entrega de cambios (*Lead Time*): Tiempo desde el *commit* del código hasta su despliegue en producción.
- Tiempo medio de recuperación (MTTR): Tiempo necesario para recuperarse de fallos.
- Tasa de fallos por cambio: Porcentaje de despliegues que causan fallos.

Esto enfatiza la importancia de establecer métricas base y monitorear continuamente las mejoras [12].

Estudios de casos de uso de grandes organizaciones como *Cisco Systems* demuestran la complejidad de transformar sistemas *legacy* de gestión de calidad para soportar organizaciones de desarrollo híbridas que usan tanto metodologías ágiles/DevOps como *Waterfall*. El marco para esta transformación está definido en seis áreas distintas: métricas, procesos, mediciones, reportes, análisis de calidad y cultura y liderazgo. Abordar estas áreas de manera integral no solo ayuda a reducir riesgos, sino que también fomenta la innovación continua y mejora la capacidad de entrega y calidad del software.

En conjunto, estos antecedentes evidencian que, si bien DevOps ha impulsado avances significativos en automatización, integración continua y entrega de software, aún persisten desafíos importantes relacionados con la integración efectiva de la calidad a lo largo de todo el pipeline, la definición de métricas consistentes y la detección temprana de defectos. Estas brechas refuerzan la necesidad de adoptar enfoques más integrales y sistemáticos de calidad continua, como el propuesto en este trabajo, el cual busca alinear prácticas de automatización, análisis y medición dentro del ciclo de vida DevOps para mejorar la capacidad de entrega y la calidad del software.

2.3 METODOLOGÍA

La metodología para seguir se basa en la investigación científica experimental.

1. Revisión bibliográfica con el objetivo de identificar los procedimientos y herramientas comúnmente utilizadas.
2. Análisis del contexto y diagnóstico inicial, utilizándolo de base para medir los resultados obtenidos posteriormente
3. Diseño del proceso de aseguramiento de calidad a implementar, adaptando las buenas prácticas encontradas a las necesidades y limitaciones del contexto del proyecto.
4. Implementación y experimentación, donde se recolectan métricas y resultados cuantitativos para evaluar el impacto del proceso implementado.
5. Evaluación empírica de resultados
6. Conclusiones y trabajos futuros

2.4 RESULTADOS ESPERADOS DEL PROYECTO

Se espera tener un impacto positivo en el proyecto de software elegido, potencialmente detectando defectos antes de lanzamientos, aumentando la candencia de estos y la confianza de implementación de cambios en el sistema. Logrando así un entorno automatizado y con la posibilidad de expandir y mantener a medida que el proyecto vaya creciendo.

3. PLANEAMIENTO DEL PROYECTO

Actividad	Inicio	Fin	Duración en días
Investigación bibliográfica y del estado del arte	01/05/26	15/06/26	45
Aplicación y análisis de prácticas, herramientas y enfoques identificados	16/06/26	31/07/26	45
Análisis de requerimientos, riesgos y definición del enfoque de calidad	01/08/26	15/09/26	45
Diseño del proceso de Calidad Continua y del pipeline CI/CD	16/09/26	31/10/26	45
Implementación de pruebas automatizadas y herramientas de análisis estático	01/11/26	15/01/27	75
Integración y ejecución del pipeline CI/CD con métricas	16/01/27	28/02/27	45
Evaluación y análisis de resultados obtenidos	01/03/27	31/03/27	30
Ajustes y optimización del proceso propuesto	01/04/27	30/04/27	30
Redacción del documento final de tesis	01/02/27	30/04/27	90

4. TEMARIO TENTATIVO DEL REPORTE TÉCNICO FINAL

- **Introducción**
- **Marco Teórico y Estado del Arte**
- **Diseño del Proceso de QA Integrado**
- **Implementación y resultados**
- **Conclusiones**

REFERENCIAS BIBLIOGRÁFICAS

1. International Software Testing Qualifications Board (ISTQB). Certified Tester Foundation Level Syllabus, version 4.0 [Internet]. 2024 Nov [cited 2025 Aug 11]. Available from: https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf
2. Gokarnaa M, Singhb R. DevOps: A Historical Review and Future Works [Internet]. arXiv preprint arXiv:2012.06145; 2020 [cited 2025 Aug 12]. Available from: <https://arxiv.org/abs/2012.06145>
3. Khomh F, Adams B, Dhaliwal T, Zou Y. Understanding the impact of rapid releases on software quality: The case of Firefox. *Empir Software Eng*. 2014; Early view. doi:10.1007/s10664-014-9308-x
4. Mantyla, M. V., Khomh, F., Adams, B., Engstrom, E., & Petersen, K. (2013). On Rapid Releases and Software Testing. *IEEE International Conference on Software Maintenance*, 312, 312-322. DOI: 10.1109/ICSM.2013.13.
5. Costa, D., McIntosh, S., Treude, C. et al. The impact of rapid release cycles on the integration delay of fixed issues. *Empir Software Eng* 23, 835–904 (2018). <https://doi.org/10.1007/s10664-017-9548-7>
6. Kerzazi, N., & Khomh, F. (2014). Factors Impacting Rapid Releases: An Industrial Case Study. In *Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '14)*, Torino, Italy, September 18-19, 2014 (pp. 180-190). ACM. <https://doi.org/10.1145/2652524.2652589>
7. A. Mishra and Z. Otaiwi, "DevOps and software quality: A systematic mapping," *Computer Science Review*, vol. 38, p. 100308, Nov. 2020
8. A. Alnafessah, A. U. Gias, R. Wang, L. Zhu, G. Casale, and A. Filieri, "Quality-Aware DevOps Research: Where Do We Stand?," *IEEE Access*, vol. 9, pp. 44476–44489, 2021
9. M. M. A. Ibrahim, S. M. Syed-Mohamad, and M. H. Husin, "Managing Quality Assurance Challenges of DevOps through Analytics," in *Proceedings of the 2019 8th International Conference on Software and Computer Applications*, ACM, Feb. 2019, pp. 194–198
10. J. Gmeiner, R. Ramler, and J. Haslinger, "Automated testing in the continuous delivery pipeline: A case study of an online company," in *2015 IEEE Eighth International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, IEEE, Apr. 2015, pp. 1–6
11. F. Zampetti, S. Scalabrino, R. Oliveto, G. Canfora, and M. Di Penta, "How Open Source Projects Use Static Code Analysis Tools in Continuous Integration Pipelines," in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*, IEEE, May 2017, pp. 334–344
12. P. Batra and A. Jatain, "Measurement Based Performance Evaluation of DevOps," in *2020 International Conference on Computational Performance Evaluation (ComPE)*, IEEE, Jul. 2020, pp. 757–760